# FGS EC2 — Full setup, bootstrap, CD, and initial testing

Complete runbook for **dev only**: one EC2 instance, configure GitHub/AWS, bootstrap once, deploy Setup / User / nginx from ECR via CD.

**Scope:** test/main branches and qa/prod GitHub Environments are **not** configured yet — add them when you expand beyond dev.

**Region example:** `us-east-1`
**Account example:** `286093098927`
**ECR repository:** `fgs/dockers`
**GitHub repo example:** `nathshivfsm-sys/fgs-coreapi`

---

## 1. Architecture

```
Internet
   │
   ▼
Application Load Balancer (optional, port 80/443)
   │
   ▼
EC2 instance (single host, e.g. dev-rabbitmq)
   │
   ├── redis          (Docker)
   ├── rabbitmq       (Docker — password from /opt/fgs/.env)
   ├── setup-service  (ECR tag setup-dev)
   ├── user-service   (ECR tag user-dev)
   └── nginx          (ECR tag nginx-dev, port 80)

GitHub Actions (merge to dev)
   ├── Build → push ECR
   └── CD → SSM SendCommand → /opt/fgs/deploy-service.sh
```

| Component | Count (dev) |
|---|---|
| EC2 instances | **1** (all services as containers) |
| ECR repositories | **1** (`fgs/dockers`) |
| Image tags per channel | `setup-dev`, `user-dev`, `nginx-dev` |

---

## 2. Prerequisites

Before you start, ensure you have:

| Item | Notes |
|---|---|
| AWS account access | EC2, ECR, IAM, SSM, optional ALB |
| GitHub repo admin | Secrets, variables, environments |
| RDS (or Postgres) | Connection strings for Setup and User databases |
| `glo.GloCredential` data | Global RABBITMQ username/password (Setup reads at startup) |
| GitHub Actions workflows | On `dev` branch: `build-setup.yml`, `build-user.yml`, `build-nginx.yml`, `reusable-deploy-ec2.yml` |

---

## 3. AWS — ECR repository

1. Open **Amazon ECR** → **Private registry** → **Create repository**.
2. Repository name: **`fgs/dockers`**
3. Create the repository.
4. Note the registry URI:
   `286093098927.dkr.ecr.us-east-1.amazonaws.com/fgs/dockers`

Images are tagged: `setup-dev`, `user-dev`, `nginx-dev` (and version-specific tags from CI).

---

## 4. AWS — IAM role for EC2 (instance profile)

The EC2 host must use SSM (for CD) and pull images from ECR.

### 4.1 Create role

1. **IAM** → **Roles** → **Create role**.
2. Trusted entity: **AWS service** → **EC2**.
3. Attach managed policy: **`AmazonSSMManagedInstanceCore`**.
4. Role name: **`fgs-dev-ec2-role`** → Create.

### 4.2 Inline policy — ECR pull

**Add permissions** → **Create inline policy** → JSON (replace `ACCOUNT_ID`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EcrAuth",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "EcrPull",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage",
        "ecr:DescribeRepositories"
      ],
      "Resource": "arn:aws:ecr:us-east-1:ACCOUNT_ID:repository/fgs/dockers"
    }
  ]
}
```

Name: **`fgs-ec2-ecr-pull`**.

---

## 5. AWS — IAM user for GitHub Actions

Use an IAM user with access keys (or OIDC role — see `GITHUB_ACTIONS_OIDC_ECR.md`).

### 5.1 Create user

1. **IAM** → **Users** → **Create user** → name: **`fgs-github-actions`**.
2. **Create inline policy** → JSON (replace `ACCOUNT_ID`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EcrAuth",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "EcrPushPull",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage",
        "ecr:PutImage",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:DescribeRepositories",
        "ecr:DescribeImages"
      ],
      "Resource": "arn:aws:ecr:us-east-1:ACCOUNT_ID:repository/fgs/dockers"
    },
    {
      "Sid": "Ec2DeployViaSsm",
      "Effect": "Allow",
      "Action": [
        "ssm:SendCommand",
        "ssm:GetCommandInvocation",
        "ssm:ListCommands",
        "ssm:ListCommandInvocations"
      ],
      "Resource": [
        "arn:aws:ssm:us-east-1::document/AWS-RunShellScript",
        "arn:aws:ec2:us-east-1:*:instance/*"
      ]
    }
  ]
}
```

1. **Security credentials** → **Create access key** → **Application running outside AWS**.
2. Save **Access key ID** and **Secret access key**.

---

# 6. AWS — Security groups

## 6.1 EC2 security group (`fgs-dev-ec2-sg`)

| Direction | Type | Port | Source |
|---|---|---|---|
| Inbound | HTTP | 80 | ALB security group (or your IP for testing) |
| Outbound | All traffic | All | `0.0.0.0/0` (SSM + ECR need HTTPS 443) |

No SSH required if you use Session Manager.

## 6.2 ALB security group (if using ALB)

| Direction | Type | Port | Source |
|---|---|---|---|
| Inbound | HTTP/HTTPS | 80 / 443 | Internet or your CIDR |

---

# 7. AWS — Launch EC2 instance

1. **EC2** → **Launch instance**.
2. **Name:** `fgs-dev` (or keep `dev-rabbitmq` if reusing).
3. **AMI:** Amazon Linux 2023 or Ubuntu 22.04 LTS.
4. **Instance type:** `t3.medium` (minimum recommended for Docker stack).
5. **Key pair:** Optional (SSM is sufficient).
6. **Network:** Same VPC as RDS and ALB.
7. **Security group:** `fgs-dev-ec2-sg`.
8. **Advanced details** → **IAM instance profile:** `fgs-dev-ec2-role`.
9. **Launch instance**.
10. Copy **Instance ID** (e.g. `i-0abc123def456789`).

## 7.1 Verify SSM (wait 2–5 minutes)

1. **Systems Manager** → **Session Manager**.
2. Instance must appear as **Online**.

Test from your laptop:

```
aws ssm start-session --target i-YOUR_INSTANCE_ID --region us-east-1
```

---

## 8. AWS — Application Load Balancer (optional, public access)

1. **EC2 → Load Balancers → Create Application Load Balancer**.
2. **Scheme:** Internet-facing.
3. **Listener:** HTTP 80 (or HTTPS 443 with ACM certificate).
4. **Target group:**
5. Target type: **Instance**
6. Port: **80**
7. Health check path: `/nginx-health`
8. Success codes: **200**
9. Register your EC2 instance.
10. Note **ALB DNS name** for testing.

---

## 9. GitHub — Repository configuration

**Settings → Secrets and variables → Actions**

### 9.1 Secrets (access key method)

| Secret | Value |
|---|---|
| `AWS_ACCESS_KEY_ID` | From IAM user `fgs-github-actions` |
| `AWS_SECRET_ACCESS_KEY` | From IAM user |

### 9.2 Repository variables

| Variable | Example |
|---|---|
| `AWS_REGION` | `us-east-1` |
| `ECR_REPO` | `fgs/dockers` |

Optional: `PUSH_TO_ECR = true` (default behaviour).

### 9.3 GitHub Environment — `dev`

**Settings → Environments → New environment** → name: `dev`

| Setting | Value |
|---|---|
| Required reviewers | **None** (skip for now — auto-deploy) |
| Environment variable `EC2_INSTANCE_ID` | Your instance ID `i-...` |

Optional environment variable:

| Variable | Default |
|---|---|
| `FGS_COMPOSE_DIR` | `/opt/fgs` |

---

## 10. EC2 — First-time bootstrap

Bootstrap runs **once** on the instance. CD does not install Docker or copy these files.

### 10.1 Connect to the instance

```
aws ssm start-session --target i-YOUR_INSTANCE_ID --region us-east-1
```

### 10.2 Get deployment files onto the instance

#### Option A — Git clone (simplest)

```
sudo yum install -y git    # Amazon Linux
# OR: sudo apt install -y git   # Ubuntu

cd /tmp
git clone https://github.com/nathshivfsm-sys/fgs-coreapi.git fgs
cd fgs
sudo bash deployment/aws/ec2/bootstrap-ec2.sh
```

#### Option B — Copy only the ec2 folder from your workstation

Copy these files to the instance, then run bootstrap from that folder:

| File | Purpose |
|---|---|
| `deployment/aws/ec2/bootstrap-ec2.sh` | Installs Docker, creates `/opt/fgs` |
| `deployment/aws/ec2/deploy-service.sh` | CD invokes this via SSM |
| `deployment/aws/ec2/docker-compose.ec2.yml` | Defines all containers |
| `deployment/aws/ec2/nginx-http-only-entrypoint.sh` | Nginx on port 80 for ALB |

### 10.3 Files created on the instance after bootstrap

| Path | Purpose |
|---|---|
| `/opt/fgs/deploy-service.sh` | Pull one ECR image + restart one container |

| Path | Purpose |
| --- | --- |
| `/opt/fgs/docker-compose.ec2.yml` | Stack definition |
| `/opt/fgs/nginx-http-only-entrypoint.sh` | Nginx entrypoint |
| `/opt/fgs/config/setup-appsettings.json` | Setup DB connection (bootstrap placeholder) |
| `/opt/fgs/config/user-appsettings.json` | User DB connection (bootstrap placeholder) |
| `/opt/fgs/.env` | Host env (RabbitMQ broker boot, ASP.NET env, etc.) |

### 10.4 Edit `setup-appsettings.json`

```
sudo nano /opt/fgs/config/setup-appsettings.json
```

Example:

```
{
  "ConnectionStrings": {
    "FgsSetup": "Host=your-
rds.region.rds.amazonaws.com;Port=5432;Database=fgs_setup;Username=...;Password=..."
  }
}
```

Setup uses this to reach RDS **before** loading other credentials from `GloCredential`.

### 10.5 Edit `user-appsettings.json`

```
sudo nano /opt/fgs/config/user-appsettings.json
```

Example:

```
{
  "ConnectionStrings": {
    "FgsUser": "Host=your-
rds.region.rds.amazonaws.com;Port=5432;Database=fgs_user;Username=...;Password=..."
  }
}
```

User-service bootstraps DB access from this file, then loads other credentials from Setup.

### 10.6 Edit `/opt/fgs/.env`

```
sudo nano /opt/fgs/.env
```

Example:

```
FGS_CONFIG_DIR=/opt/fgs/config
ASPNETCORE_ENVIRONMENT=Development
RABBITMQ_USER=fgs
RABBITMQ_PASSWORD=YourStrongPasswordMatchingGloCredential
CREDENTIAL_DISTRIBUTION_KEY=fgs-internal-credential-distribution-key
FGS_CHANNEL=dev
```

**Important — RabbitMQ two-layer model:**

| Where | Purpose |
|---|---|
| `/opt/fgs/.env` `RABBITMQ_PASSWORD` | Starts the **RabbitMQ Docker container** (`RABBITMQ_DEFAULT_PASS`) |
| `glo.GloCredential` `Global:RABBITMQ` | Setup reads **Username/Password** at startup |

These passwords **must match**. Setup does **not** get `RabbitMq__Password` from compose (would override the credential table).

Ensure in your Setup database (`glo.GloCredential`, provider `RABBITMQ`):

- `Username` = same as `RABBITMQ_USER` (default `fgs`)
- `Password` = same as `RABBITMQ_PASSWORD`

Optional: set `Global:RABBITMQ:ConnectionUri` to `amqp://fgs:YourPassword@rabbitmq:5672/` instead of separate host/user/pass in compose.

---

## 11. Push images to ECR (first time)

Images must exist in ECR before the instance can pull them.

### Option A — Merge to `dev` (recommended)

1. Bump `<Version>` in `src/SetupService/Fgs.Setup.API/Fgs.Setup.API.csproj`.
2. Bump `<Version>` in `src/UserService/Fgs.User.API/Fgs.User.API.csproj`.
3. Bump `src/Gateway/VERSION` for nginx.
4. Merge each change to `dev` (or one PR with all three).

GitHub Actions builds and pushes:

- `fgs/dockers:setup-dev`
- `fgs/dockers:user-dev`
- `fgs/dockers:nginx-dev`

If bootstrap and `EC2_INSTANCE_ID` are ready, **deploy jobs run automatically** after each push.

### Option B — Manual workflow dispatch

**Actions** → **Build setup** → **Run workflow** → branch `dev` → enable **push_to_ecr**.

Repeat for **Build user** and **Build nginx**.

Verify in **ECR** → `fgs/dockers` → **Images** tags exist.

---

## 12. First deploy on EC2

### Option A — Let CD deploy (after bootstrap + GitHub config)

Merge version bumps to `dev`. Each workflow runs deploy via SSM.

Deploy order (dependencies): **setup** → **user** → **nginx**.

If user deploy runs before setup image exists, wait for setup deploy first, then re-run user workflow if needed.

### Option B — Manual first start (all services at once)

On the instance:

```
cd /opt/fgs
sudo ./deploy-service.sh setup-service dev fgs/dockers us-east-1
sudo ./deploy-service.sh user-service dev fgs/dockers us-east-1
sudo ./deploy-service.sh nginx dev fgs/dockers us-east-1
```

Or start infrastructure first, then apps:

```
cd /opt/fgs
docker compose -f docker-compose.ec2.yml up -d redis rabbitmq
# wait until healthy, then deploy-service commands above
```

---

## 13. CD flow (every release after bootstrap)

```
1. Developer bumps service version (csproj or Gateway/VERSION)
2. PR merged to dev
3. GitHub Actions — build job:
     - Run tests (setup/user)
     - docker build
     - docker push to ECR (only on merge to dev, not on PR)
4. GitHub Actions — deploy job (if image_pushed):
     - GitHub Environment: dev (no approval)
     - AWS auth (access keys or OIDC)
     - SSM SendCommand to EC2_INSTANCE_ID:
         sudo /opt/fgs/deploy-service.sh <service> dev fgs/dockers us-east-1
5. EC2: docker pull + docker compose up -d --no-deps <service>
```

| Branch | Channel tag | GitHub Environment | Deploy |
|---|---|---|---|
| dev | *-dev | dev | Auto |

**PR builds:** compile and test only — **no ECR push, no deploy**.

---

## 14. Initial testing

Run on the EC2 instance (SSM session) unless noted.

### 14.1 Container status

```
docker compose -f /opt/fgs/docker-compose.ec2.yml ps
```

All services should be **Up** / **healthy**.

### 14.2 Infrastructure health

```
docker exec $(docker ps -qf name=redis) redis-cli ping
# Expected: PONG

docker exec $(docker ps -qf name=rabbitmq) rabbitmq-diagnostics -q ping
# Expected: Ping succeeded
```

### 14.3 Service health (on instance)

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/nginx-health
# Expected: 200

curl -s -o /dev/null -w "%{http_code}\n" http://setup-service:5004/health
# Run from another container on same network, or:
docker exec $(docker ps -qf name=setup-service) curl -fsS http://localhost:5004/health

docker exec $(docker ps -qf name=user-service) curl -fsS http://localhost:5001/health
```

### 14.4 Through nginx (local)

```
curl -s http://localhost/nginx-health
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/swagger/setup/
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/swagger/user/
```

### 14.5 Through ALB (from your laptop)

Replace `ALB_DNS` with your load balancer DNS name:

```
curl -s http://ALB_DNS/nginx-health
curl -s -o /dev/null -w "%{http_code}\n" http://ALB_DNS/nginx-health
```

Target group should show instance **healthy**.

### 14.6 Verify ECR tags in use

```
docker inspect $(docker ps -qf name=setup-service) --format '{{.Config.Image}}'
docker inspect $(docker ps -qf name=user-service) --format '{{.Config.Image}}'
docker inspect $(docker ps -qf name=nginx) --format '{{.Config.Image}}'
```

Expected pattern:
`286093098927.dkr.ecr.us-east-1.amazonaws.com/fgs/dockers:setup-dev` (etc.)

### 14.7 Verify CD from GitHub

1. Bump User version only → merge to `dev`.
2. **Actions** → **Build user** → confirm **build** and **deploy** jobs succeed.
3. On EC2, confirm user container recreated with new image digest.

### 14.8 Logs if something fails

```
docker logs $(docker ps -qf name=setup-service) --tail 100
docker logs $(docker ps -qf name=user-service) --tail 100
docker logs $(docker ps -qf name=nginx) --tail 50
```

Common issues:

- Setup unhealthy → wrong RDS string in `setup-appsettings.json`
- RabbitMQ connection failed → `.env` password ≠ `GloCredential` RABBITMQ
- ECR pull denied → EC2 instance role missing ECR policy
- CD SSM failed → missing `EC2_INSTANCE_ID` or SSM permissions on GitHub IAM user

---

## 15. Master checklist

### AWS

- [ ] ECR repo `fgs/dockers` created
- [ ] EC2 IAM role: SSM + ECR pull
- [ ] GitHub IAM user: ECR push + SSM SendCommand
- [ ] EC2 instance launched with instance profile
- [ ] SSM Online
- [ ] Security group allows port 80 from ALB
- [ ] ALB target group → instance :80, health `/nginx-health`
- [ ] RDS reachable from EC2 security group

### GitHub

- [ ] Secrets: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`
- [ ] Variables: `AWS_REGION`, `ECR_REPO`
- [ ] Environment `dev` with `EC2_INSTANCE_ID`
- [ ] Workflow files on `dev` branch

### EC2 bootstrap (one time)

- [ ] `bootstrap-ec2.sh` run
- [ ] `setup-appsettings.json` — real FgsSetup connection string
- [ ] `user-appsettings.json` — real FgsUser connection string
- [ ] `.env` — `RABBITMQ_PASSWORD` matches GloCredential RABBITMQ
- [ ] Images in ECR (`setup-dev`, `user-dev`, `nginx-dev`)
- [ ] All containers running and healthy

### Testing

- [ ] `/nginx-health` → 200
- [ ] Setup / User health endpoints OK
- [ ] ALB health check passing

- [ ] CD deploy job succeeds on merge to `dev`

---

## 16. Reference — repo files

| Path | Role |
|---|---|
| `.github/workflows/build-setup.yml` | CI/CD for Setup |
| `.github/workflows/build-user.yml` | CI/CD for User |
| `.github/workflows/build-nginx.yml` | CI/CD for nginx |
| `.github/workflows/reusable-build-service.yml` | Shared build + ECR push |
| `.github/workflows/reusable-deploy-ec2.yml` | SSM deploy to EC2 |
| `deployment/aws/ec2/bootstrap-ec2.sh` | One-time host setup |
| `deployment/aws/ec2/deploy-service.sh` | Per-service deploy on EC2 |
| `deployment/aws/ec2/docker-compose.ec2.yml` | Container stack |
| `deployment/aws/manual-guide/GITHUB_ACTIONS_CD_EC2.md` | CD quick reference |

---

## 17. Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| Deploy skipped | No version bump or PR build only | Merge to dev with version change |
| `Set EC2_INSTANCE_ID` | Missing on GitHub Environment `dev` | Add variable |
| SSM AccessDenied | GitHub IAM missing SSM policy | Add `Ec2DeployViaSsm` policy |
| SSM Failed: no such file | Bootstrap not run | Run `bootstrap-ec2.sh` |
| ECR pull denied on EC2 | Instance role missing ECR pull | Fix EC2 IAM policy |
| Setup crash loop | Bad RDS connection string | Fix `setup-appsettings.json` |
| RabbitMQ auth error | Password mismatch | Align `.env` and GloCredential |
| nginx 502 | Upstreams not healthy | Fix setup/user first |
| Instance not in Session Manager | No SSM role or no outbound 443 | Fix instance profile / SG |

---

*Document version: EC2 single-instance stack with external credential table for Setup RabbitMQ auth.*